Which type of scaling is described below?
When the CPU utilisation of the server crosses a threshold, add a new server to the backend.

47 users have participated

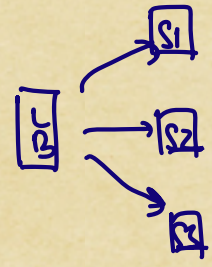
A	Vertical Scaling.	6%
✓ B	Horizontal Scaling.	94%
C	Mixed Scaling.	0%
D	Cross Scaling.	0%

Which of the following is not a responsibility of the load balancer?

50 users have participated

A	Distributing requests to the different servers.	8%
B	Checking and storing the status of the servers.	12%
✓ C	Resolving the IP address from the domain name.	80%

(correct)



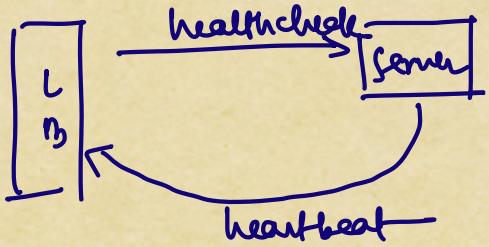
Which of the following is not a mechanism to track the health of a server?

55 users have participated

A	Heartbeat.	15%
✓ B	Sharding.	84%
C	Healthcheck.	2%

→ Server sending healthcheck to LB

→ LB checking health for server



The process of dividing your dataset into smaller and storing it on different database instances is known as?

55 users have participated

A	Caching.	0%
✓ B	Sharding.	67%
C	Consistent Hashing.	33%
D	Inconsistent Hashing.	0%

→ Sharding / Partitioning

data division

→ algorithm / way to divide / distribute the data.

(phoneNo.) % 3 →

- → DB1 (0)
- → DB2 (1)
- → DB3 (2)

→ Agenda

i) Caching

ii) CDNs.

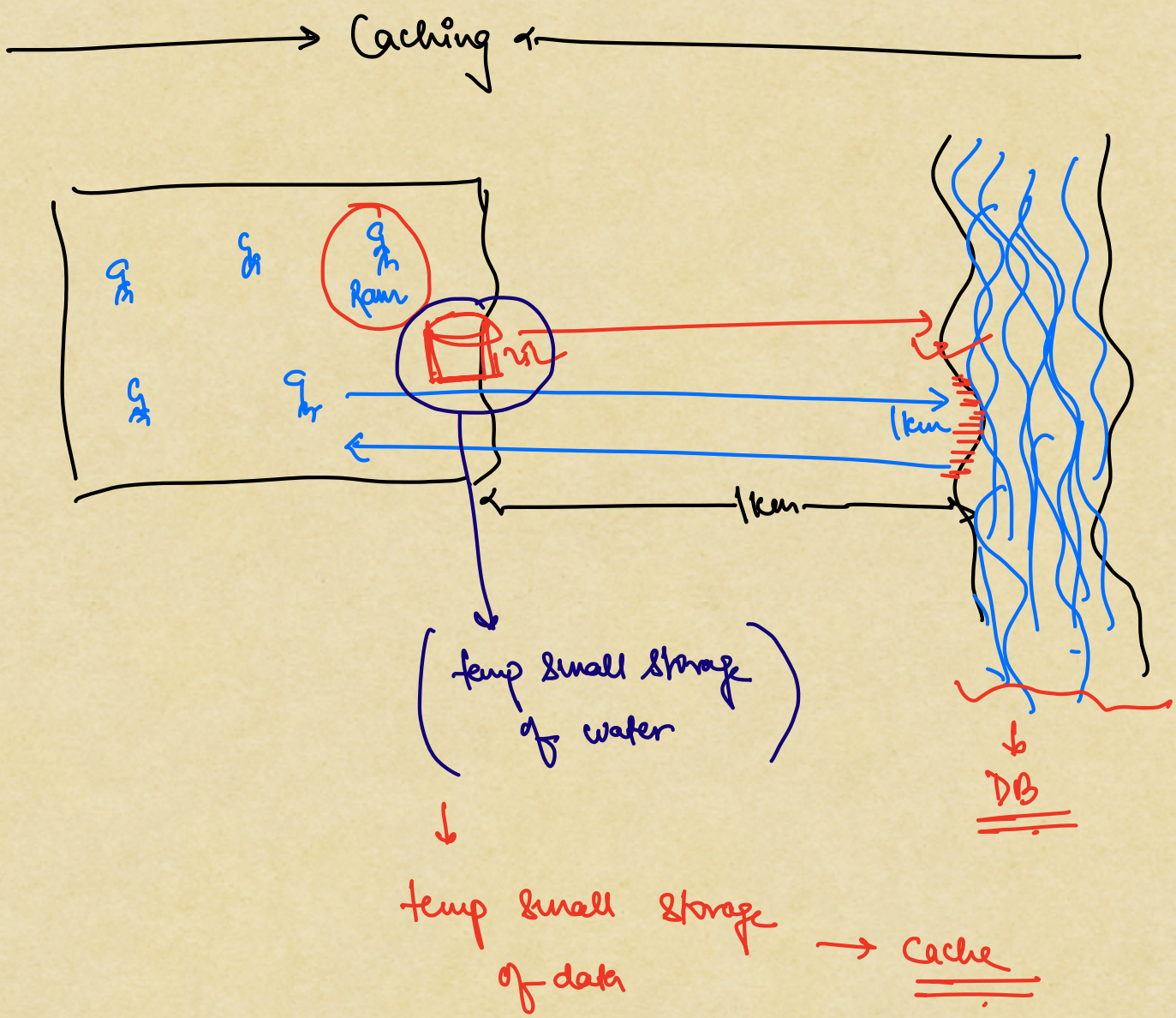
iii) Global vs local cache

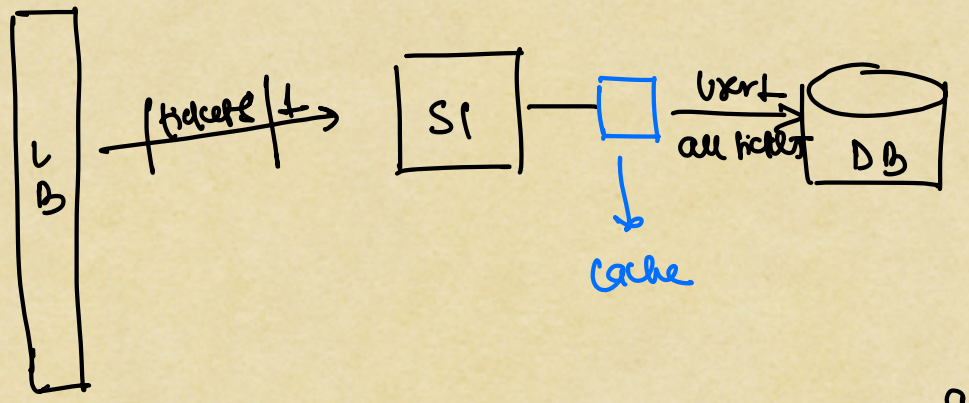
iv) Invalidation (cache)

H.W

1) Consistent hashing can be applied at both places at load balancer and as well between application server and databases ?

2) What is stickiness?

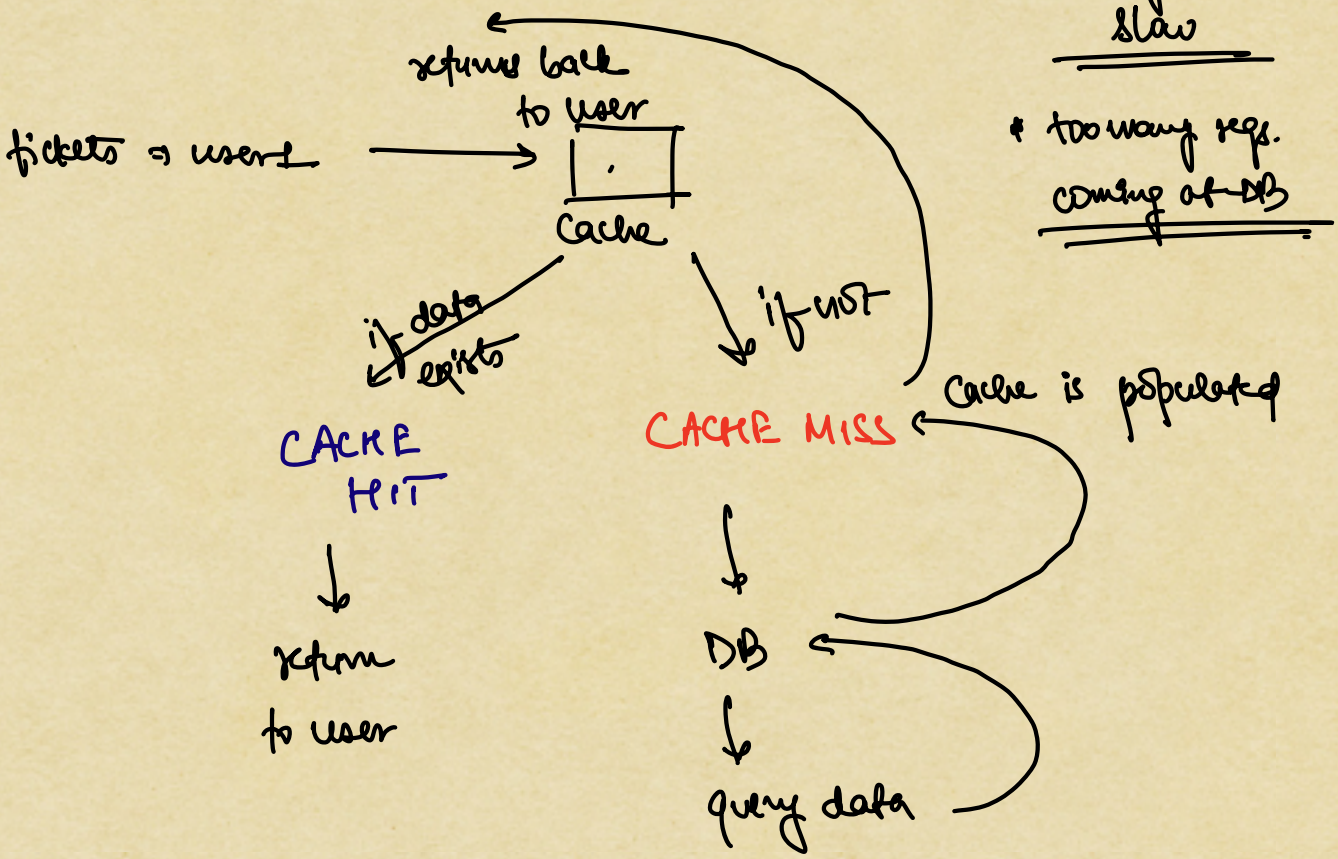




10M records

* query is slow

* too many reqs. coming at DB



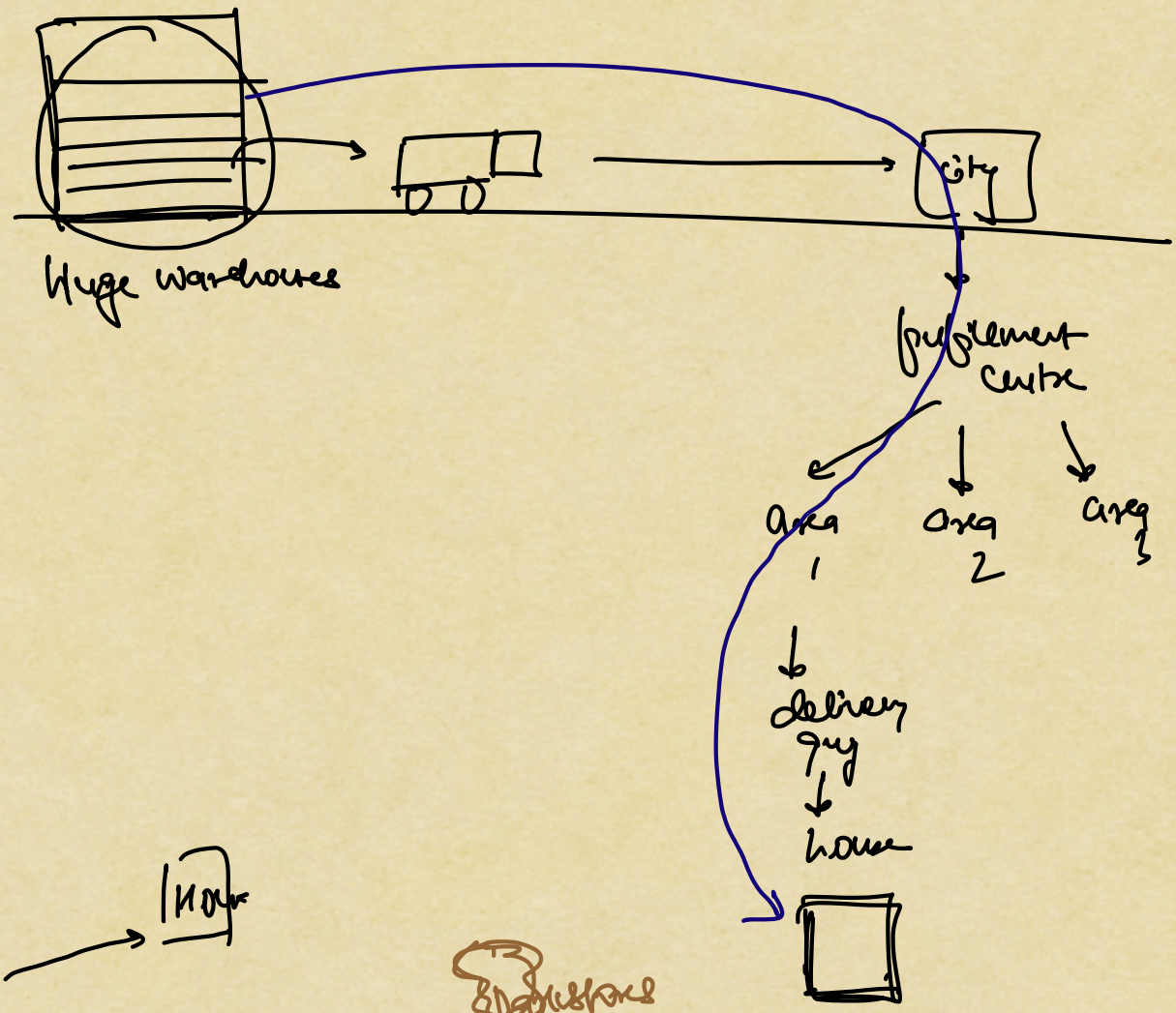
Cache → high performance

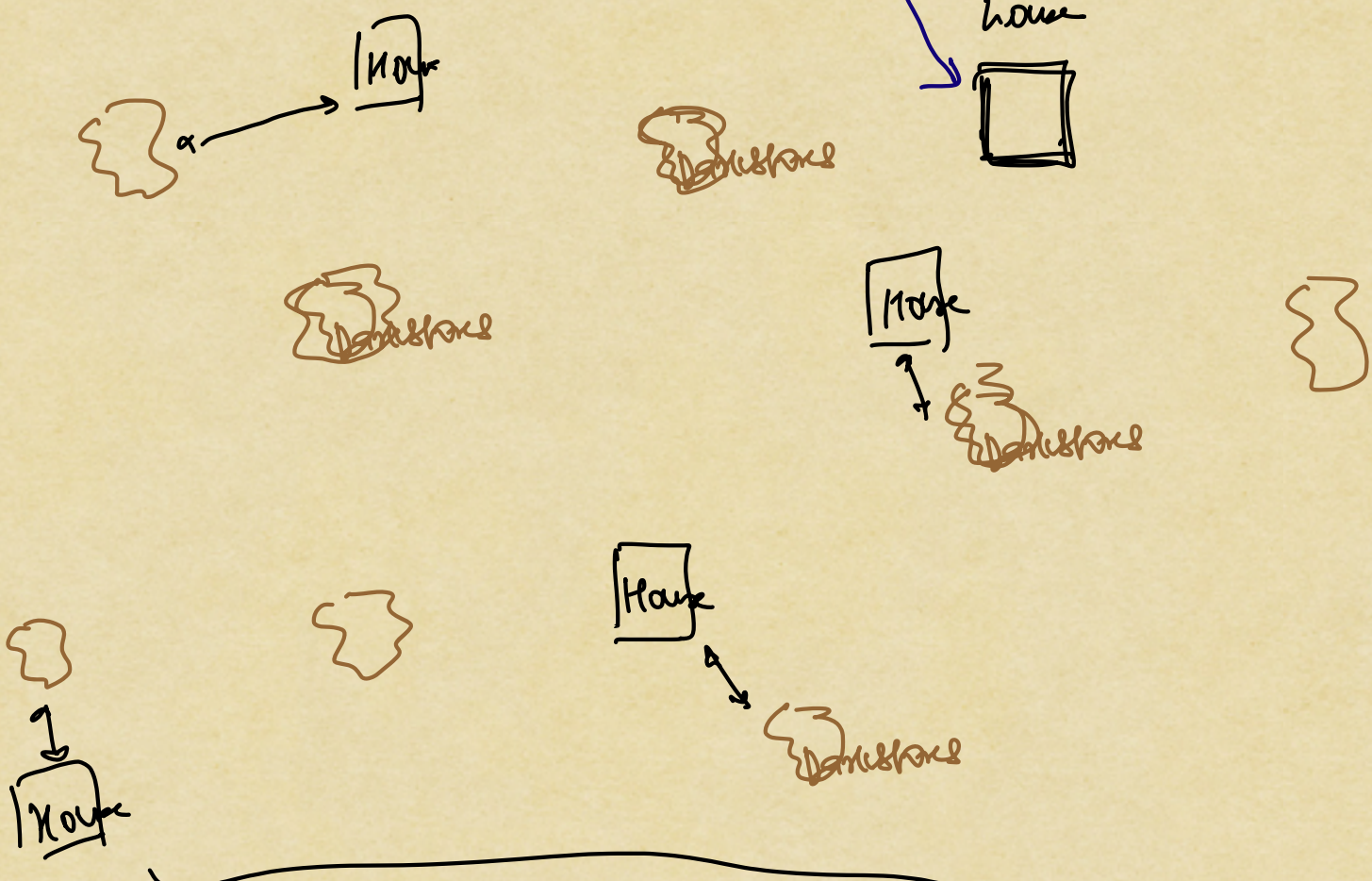
H/W → high in cost

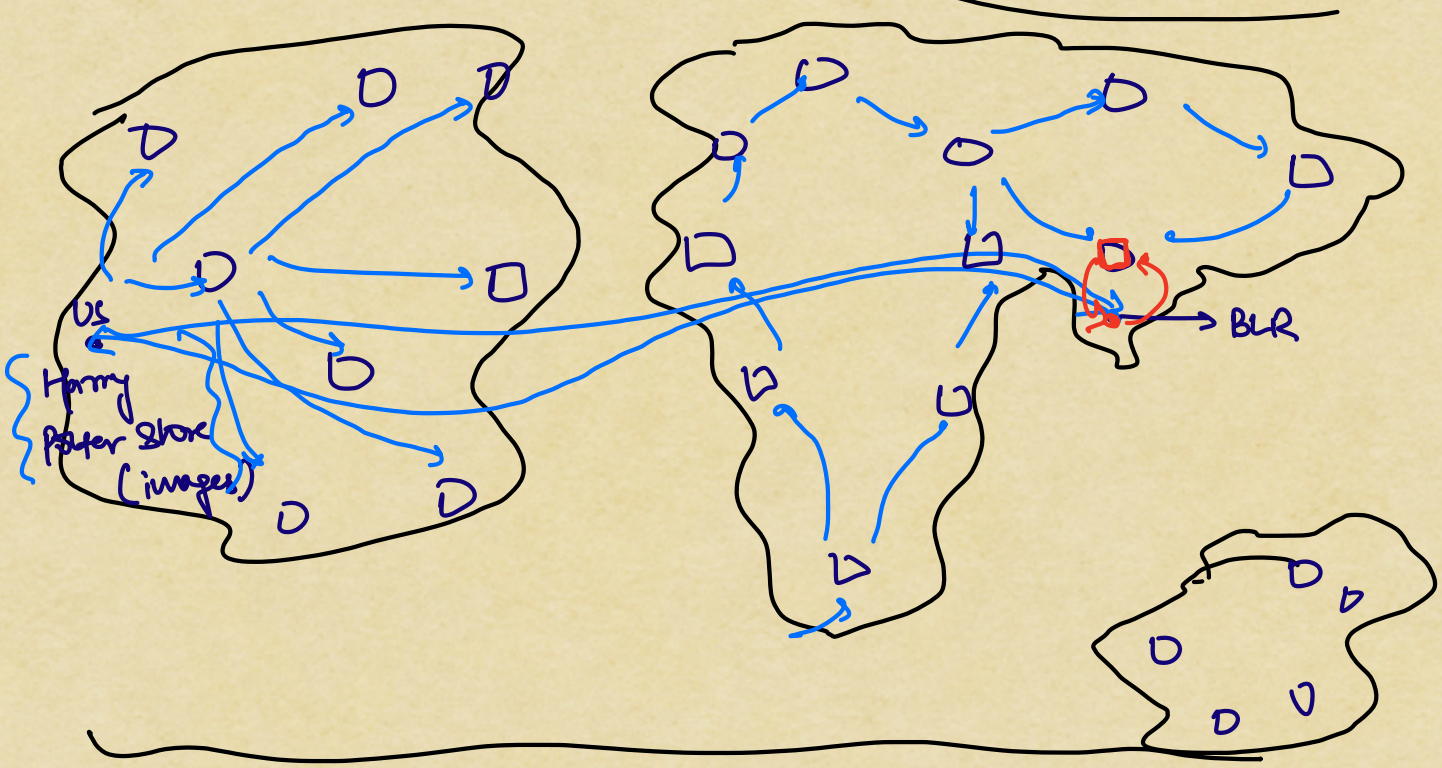
→ CDNs

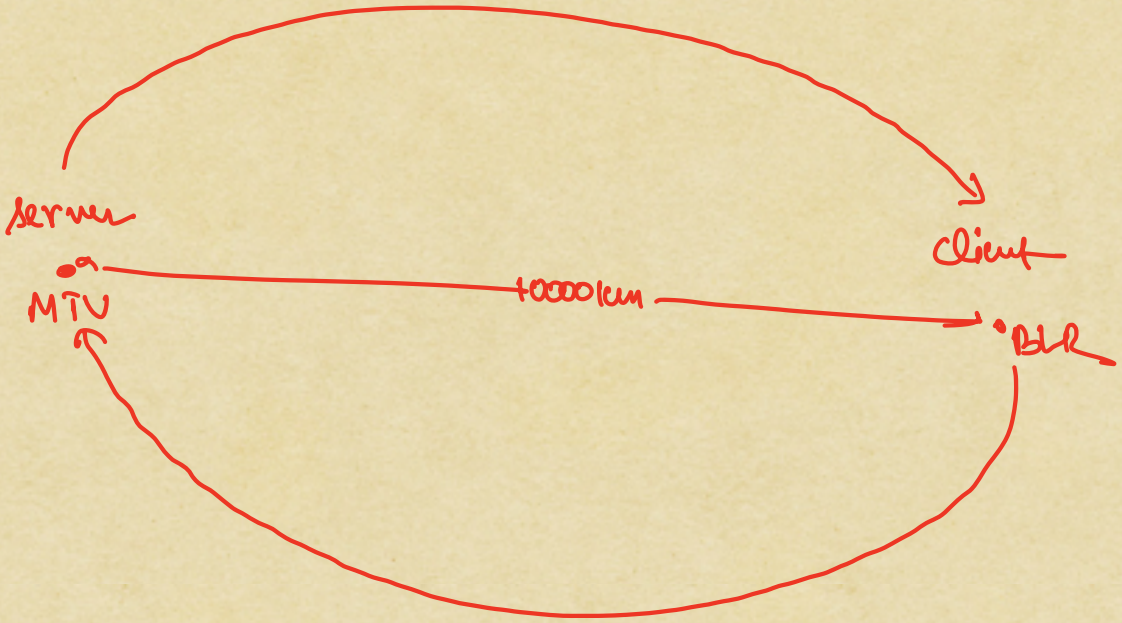
↓

Content Delivery Network

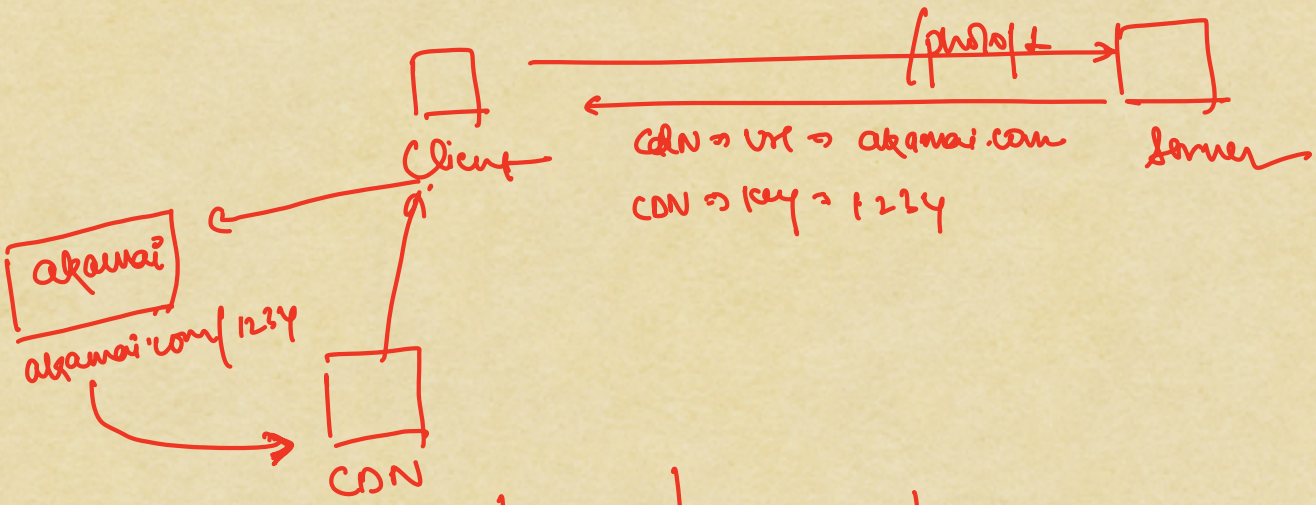








20000 km ⇒ 150 ms



key	value
1234	photo

⇒ Types of caches

i) local cache

ii) global cache

i) local cache

⇒ Storage in the server can be used as a cache

⇒ Guava cache

⇒ In-memory ⇒ Hashmap

⇒ Caffeine cache

⇒ Springboot cache

→ advantages

→ extremely fast

→ no n/w lag

→ RAM is very fast

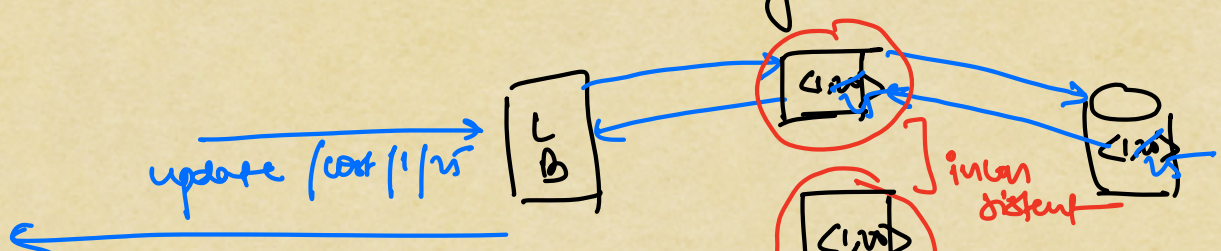
(memory lookup is in nanosecs.)

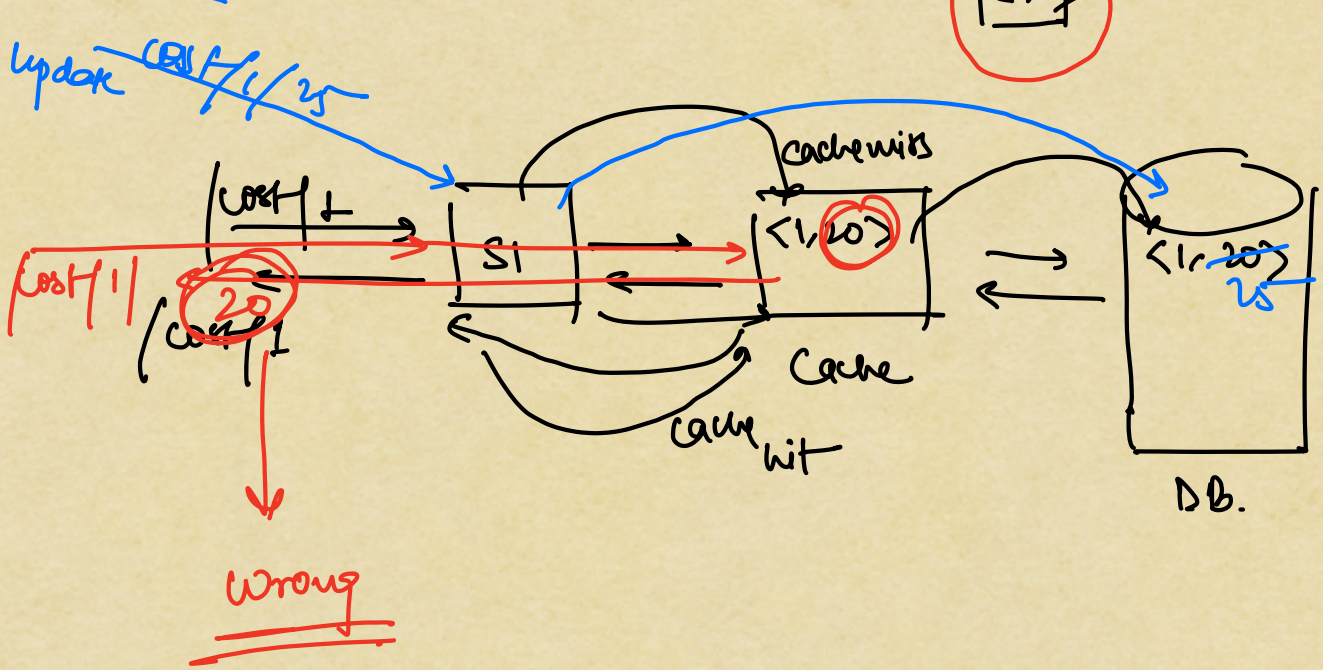
→ disadvantages

→ cache duplication

if we have 10 servers, we might end up caching the same thing 10 times.

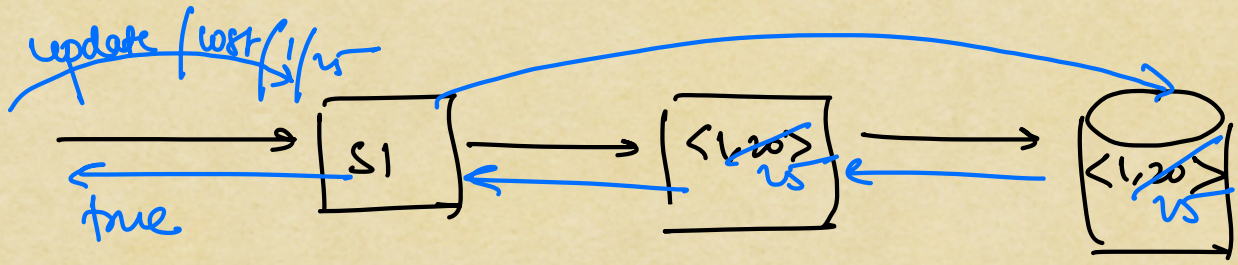
→ cache inconsistency





Wrong

1) update the cache when updating data



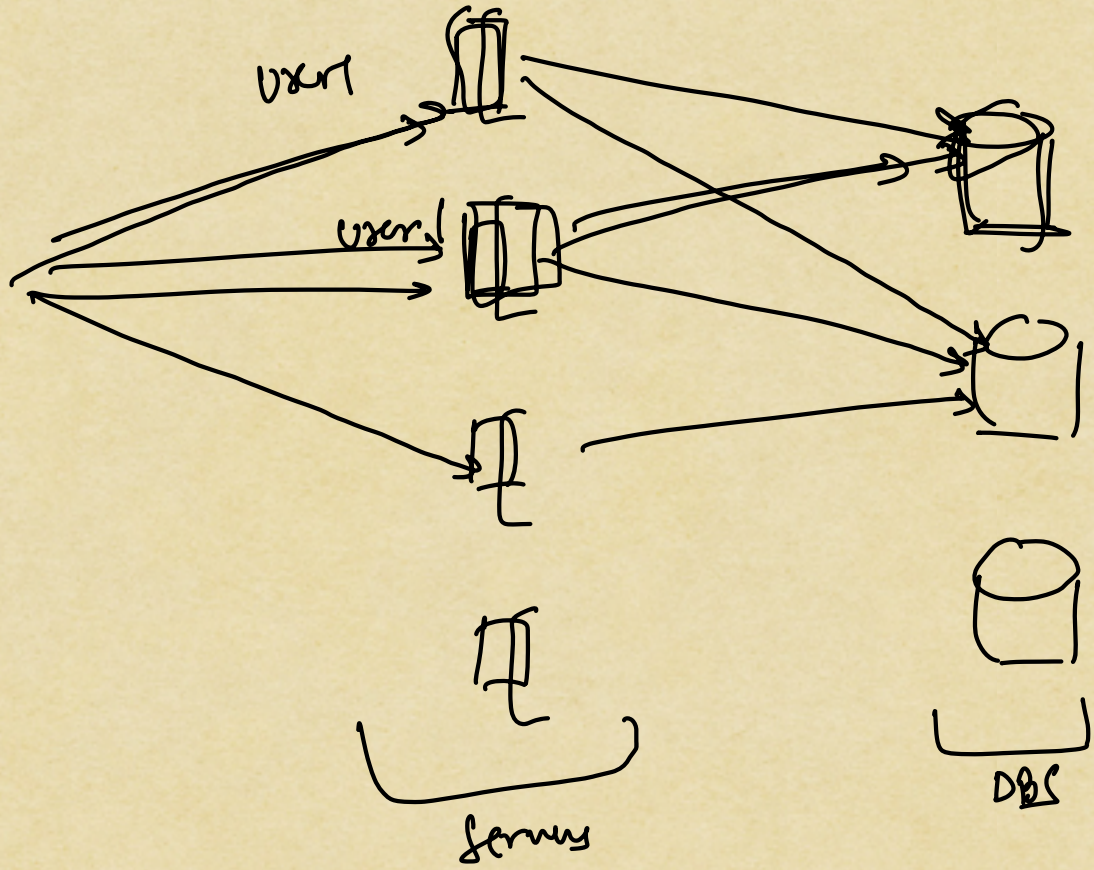
→ Global Cache | Distributed Cache

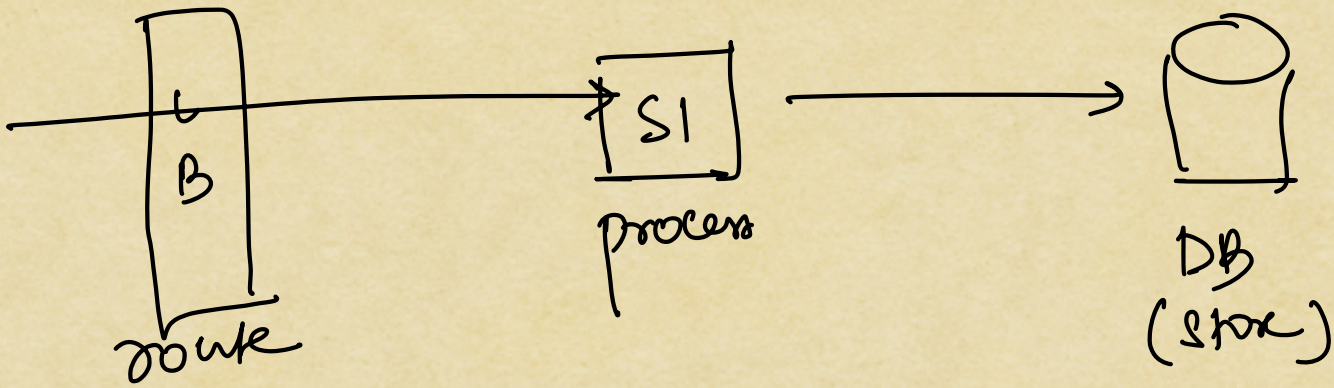
→ Redis

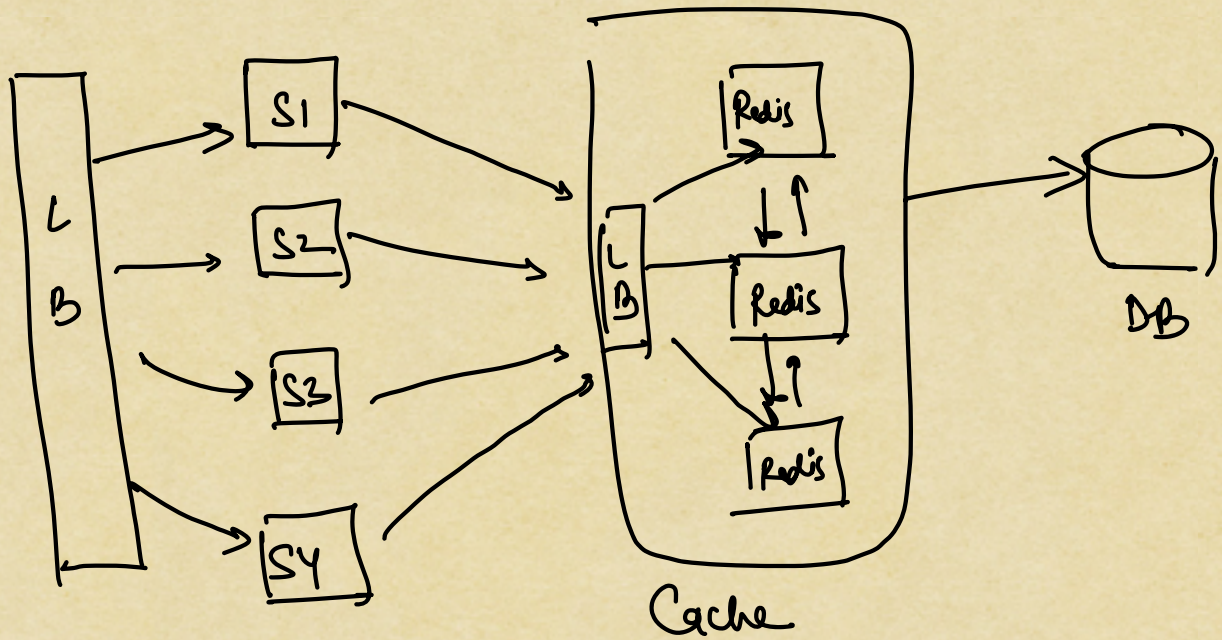
→ Memcached

→ AWS DynamoDB DAX → Cache

↓
DB


User1
User2
Servers
DBS


L
B
route
S1
process
DB
(store)


L
B
S1
S2
S3
S4
L
B
Redis
Redis
Redis
Cache
DB

→ advantages

→ consistency

data remains consistent at cache level

→ no duplication

→ disadvantages

→ cost

→ n/w hop (slowing down resp. time)

→ slower than local cache

→ Challenges with caching

Two most difficult problems

— naming things

— cache invalidation

i) Cache has limited cache

ii) Cache has invalid / stale data

→ cache is not source of truth, DB is source of truth

→ Strategies for cache invalidation / eviction :-

1) TTL (time to live)

eg ⇒ 5 mins

every entry would be deleted after 5 mins.

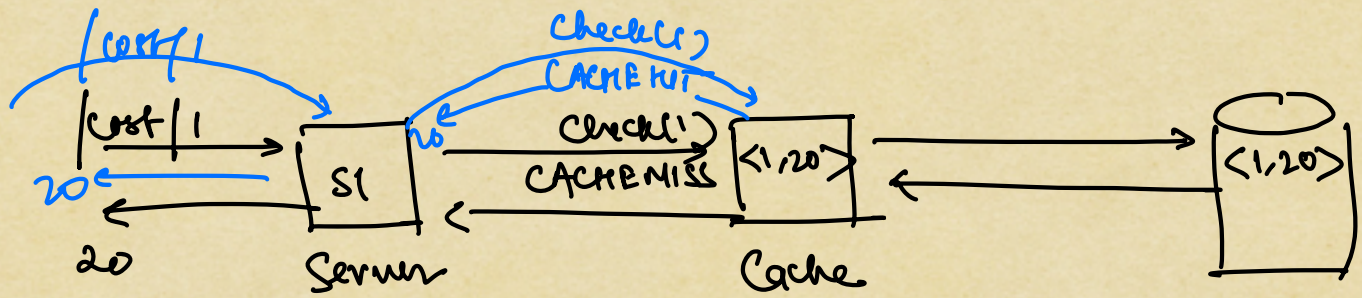
i) Cache Aside (lazy loading) → famous

ii) write through cache

iii) write back cache

iv) write around cache

1) Cache Aside



⇒ browser

1) → google search → link → fresh load

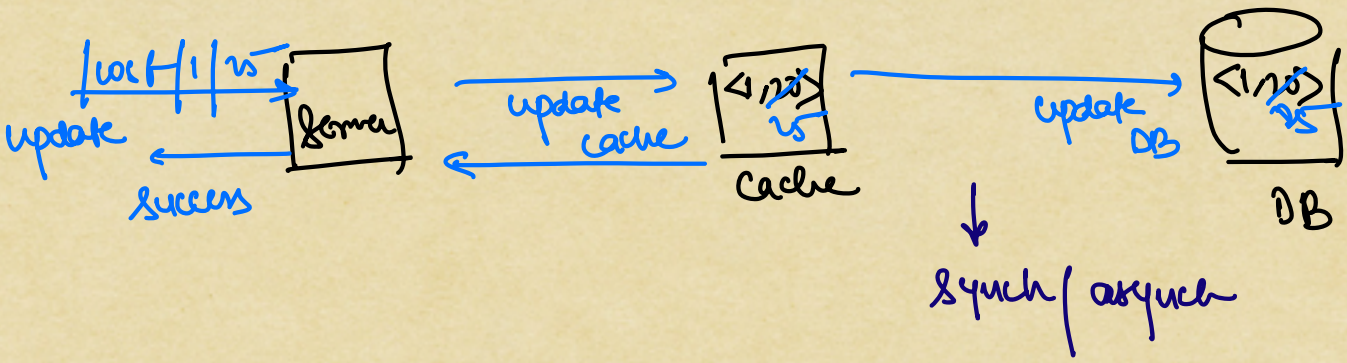
2) → google search → link → cache ← old stale

⇒ (Cache aside + TTL) general appln of cache

data possible

2) Write through cache

Overall flow similar to cache aside



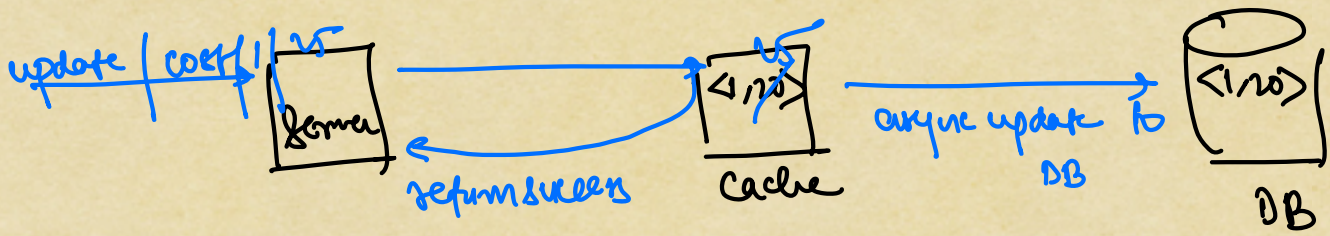
ex ⇒ google docs (autosaves.

↓

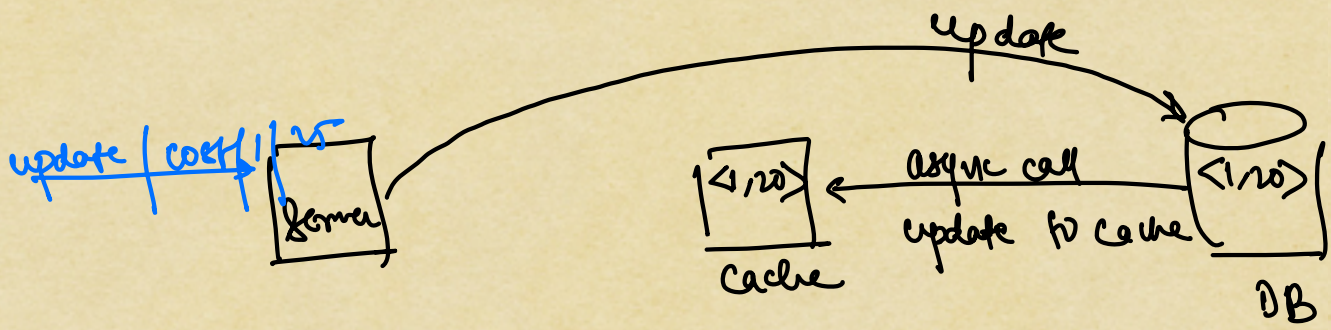
cached version in browser

3) Write back (asyn flow of write through)

Overall flow similar to cache aside



4) write around cache



CDN updates

⇒ evicting into a cache

- ⇒ cache becomes full
- ⇒ which data to delete / replace
- ⇒ algos
  - ⇒ LRU (least recently used)
  - ⇒ LFU (least frequently used)
  - ⇒ LIFO
  - ⇒ FIFO
  - ⇒ MRU (most recently used)